Wall-E: ESP32 micro-ROS Documentation

Release 0.1.0

Lydia Chheng

Apr 03, 2026

1 Getting Started **1**
1.1 Quick Start Guide . . . 1
1.2 Initial Setup . . . 1
1.3 Building the ESP32 Firmware . . . 1
1.4 Next Steps . . . 2

2 ESP32 Controller **3**
2.1 Overview . . . 3
2.2 Hardware Configuration . . . 3
2.3 Build and Flash . . . 3
2.4 Code Structure . . . 4
2.5 ROS Publishers . . . 6
2.6 WiFi Configuration . . . 6
2.7 Troubleshooting . . . 7
2.8 Future Enhancements . . . 7

3 Architecture **9**
3.1 System Overview . . . 9
3.2 Component Descriptions . . . 9
3.3 Communication Flow . . . 9
3.4 Data Types . . . 10

4 Troubleshooting **11**
4.1 Common Issues and Solutions . . . 11
4.2 Using Debug Tools . . . 13
4.3 Getting Help . . . 13

Chapter 1

Getting Started

1.1 Quick Start Guide

1.1.1 Prerequisites

- Docker and Docker Compose
- ESP-IDF v5.0+ (for ESP32 firmware)
- Git

1.2 Initial Setup

1. Clone the repository:

```
git clone https://github.com/c-lydia/wall_e.git
cd wall_e/micro_ros_ws
```

2. Start the Docker Compose environment:

```
docker compose up
```

3. Build your ROS 2 packages:

```
# In the workspace container
colcon build
```

1.3 Building the ESP32 Firmware

```
cd firmware/dev_ws
idf.py build
idf.py flash
idf.py monitor
```

1.4 Next Steps

- Read the ESP32 Controller documentation
- Explore the Architecture overview
- Check Troubleshooting for common issues

Chapter 2

ESP32 Controller

The ESP32 controller is the main FreeRTOS application running on the ESP32, handling sensor I/O and micro-ROS communication.

2.1 Overview

The controller manages:

- **IMU (MPU6050)**: Accelerometer and gyroscope data via I2C
- **Ultrasonic Sensor (HC-SR04)**: Range/distance measurements
- **micro-ROS Agent**: UDP communication with the host ROS 2 system

2.2 Hardware Configuration

2.2.1 I2C (IMU)

- **SCL (GPIO 22)**: Serial Clock
- **SDA (GPIO 21)**: Serial Data
- **Device Address**: 0x68 (MPU6050)
- **Frequency**: 100 kHz

2.2.2 Ultrasonic Sensor

- **Trigger Pin**: GPIO 9
- **Echo Pin**: GPIO 10

2.3 Build and Flash

2.3.1 Prerequisites

- ESP-IDF v5.0+
- micro-ROS firmware libraries
- FreeRTOS patched for micro-ROS

2.3.2 Build Command

```
cd firmware/dev_ws
idf.py build
```

2.3.3 Flash Command

```
idf.py flash
```

2.3.4 Monitor Serial Output

```
idf.py monitor
```

2.4 Code Structure

```
firmware/custom/esp32_controller/
├── app.c                  # Main application logic
└── app-colcon.meta        # Build configuration
```

2.4.1 Main Functions

The controller implements several key functions for operation:

app_main() - Entry Point

```
void app_main(void) {
    ESP_LOGI(TAG, "Starting Wall-E ESP32 Controller");

    // Initialize WiFi
    wifi_init();

    // Initialize I2C for IMU
    i2c_master_init();

    // Initialize GPIO for ultrasonic
    ultrasonic_init();

    // Initialize micro-ROS
    rcl_allocator_t allocator = rcl_get_default_allocator();
    rclc_support_init(&support, 0, NULL, &allocator);

    // Create node
    rclc_node_init_default(&node, "esp32_node", "", &support);

    // Create publishers
    rclc_publisher_init_default(
        &imu_pub,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(sensor_msgs, msg, Imu),
        "/imu"
    );

    // Create timer for publishing at 10Hz
    rclc_timer_init_default(&timer, &support, RCL_MS_TO_NS(100), timer_callback);

    // Start executor
    rclc_executor_init(&executor, &support.context, 1, &allocator);
```

```
    rclc_executor_add_timer(&executor, &timer);

    ESP_LOGI(TAG, "Wall-E ESP32 Controller initialized successfully");
}
```

timer_callback() - Sensor Publishing Loop

```
void timer_callback(rcl_timer_t *timer, int64_t last_call_time) {
    if (timer != NULL) {
        // Read IMU data
        mpu6050_accel_t accel;
        mpu6050_gyro_t gyro;
        mpu6050_read_accel(&accel);
        mpu6050_read_gyro(&gyro);

        // Publish IMU message
        sensor_msgs__msg__Imu imu_msg = {0};
        imu_msg.linear_acceleration.x = accel.x;
        imu_msg.linear_acceleration.y = accel.y;
        imu_msg.linear_acceleration.z = accel.z;
        imu_msg.angular_velocity.x = gyro.x;
        imu_msg.angular_velocity.y = gyro.y;
        imu_msg.angular_velocity.z = gyro.z;

        rcl_publish(&imu_pub, &imu_msg, NULL);

        // Read and publish ultrasonic range
        float distance_m = ultrasonic_read_distance_cm() / 100.0f;
        sensor_msgs__msg__Range range_msg = {0};
        range_msg.range = distance_m;
        range_msg.min_range = 0.02f;
        range_msg.max_range = 4.0f;

        rcl_publish(&range_pub, &range_msg, NULL);
    }
}
```

WiFi Initialization

```
void wifi_init(void) {
    tcpip_adapter_init();
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    wifi_config_t wifi_config = {
        .sta = {
            .ssid = WIFI_SSID,
            .password = WIFI_PASS,
        },
    };

    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
    ESP_ERROR_CHECK(esp_wifi_set_config(ESP_IF_WIFI_STA, &wifi_config));
    ESP_ERROR_CHECK(esp_wifi_start());

    ESP_LOGI(TAG, "WiFi initialized, connecting to SSID: %s", WIFI_SSID);
}
```

I2C Master Initialization

```
void i2c_master_init(void) {
    const i2c_config_t i2c_config = {
        .mode = I2C_MODE_MASTER,
        .sda_io_num = I2C_MASTER_SDA_IO,  // GPIO 21
        .scl_io_num = I2C_MASTER_SCL_IO,  // GPIO 22
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = I2C_MASTER_FREQ_HZ,  // 100 kHz
    };

    ESP_ERROR_CHECK(i2c_param_config(I2C_MASTER_NUM, &i2c_config));
    ESP_ERROR_CHECK(i2c_driver_install(I2C_MASTER_NUM, i2c_config.mode, 0, 0, 0));
}
```

Key Constants

```
// WiFi Configuration
#define WIFI_SSID               "your_wifi_ssid"
#define WIFI_PASS               "your_wifi_password"

// I2C Configuration
#define I2C_MASTER_NUM          0
#define I2C_MASTER_SDA_IO       21
#define I2C_MASTER_SCL_IO       22
#define I2C_MASTER_FREQ_HZ      100000

// GPIO Configuration
#define ULTRASONIC_TRIGGER_GPIO 9
#define ULTRASONIC_ECHO_GPIO    10

// micro-ROS Configuration
#define RCL_MAX_NODES           1
#define RCL_MAX_PUBLISHERS      2
#define RCL_HISTORY_DEPTH       4
```

2.5 ROS Publishers

Topic	Message Type	Description
`/imu`	`sensor_msgs::msg::Imu`	6-axis IMU data (accel + gyro)
`/range`	`sensor_msgs::msg::Range`	Ultrasonic distance measurement

2.6 WiFi Configuration

```
#define WIFI_SSID "your_wifi_ssid"
#define WIFI_PASS "your_wifi_password"
```

Warning Update WiFi credentials in `app.c` before building. The placeholder values above are for documentation only.

2.7 Troubleshooting

2.7.1 I2C Communication Fails

- Check pull-up resistors on SCL/SDA
- Verify I2C frequency (100 kHz default)
- Confirm MPU6050 address matches (0x68)

2.7.2 Ultrasonic Measurements Are Erratic

- Ensure GPIO 9/10 are not used elsewhere
- Check sensor power supply (5V recommended)
- Verify echo pin is connected to an ADC-capable GPIO

2.7.3 micro-ROS Connection Issues

- Confirm WiFi SSID/password are correct
- Check micro-ROS agent is running (`docker compose up`)
- Verify UDP port 9999 is accessible
- Ensure ESP32 can reach the host machine on the network

2.8 Future Enhancements

- [ ] Add magnetometer (compass) support
- [ ] Implement sensor fusion (Kalman filter)
- [ ] Add servo/motor control
- [ ] SD card logging for offline recording

Chapter 3

Architecture

3.1 System Overview

The micro_ros_ws system consists of multiple interconnected components:

```
┌──────────────────────────────────────┐
│          ROS 2 Humble (Host)         │
│  - micro-ROS Agent (Port 9999 UDP)   │
│  - Sensor subscribers                │
└──────────────────────────────────────┘
                ▲
                │ UDP
                ▼
┌──────────────────────────────────────┐
│           ESP32 + FreeRTOS           │
│  - IMU (MPU6050) via I2C             │
│  - Ultrasonic Sensor (HC-SR04)       │
│  - micro-ROS Client                  │
└──────────────────────────────────────┘
```

3.2 Component Descriptions

3.2.1 Host System (Docker)

- **workspace container**: Builds and runs ROS 2 packages
- **micro_ros_agent**: Handles communication with ESP32
- **Volume mounts**: For persistent src/, build/, install/, docs/

3.2.2 ESP32 Firmware

- **FreeRTOS**: Real-time operating system
- **micro-ROS Client**: Lightweight ROS 2 client library
- **Sensor drivers**: I2C for IMU, GPIO for ultrasonic sensor

3.3 Communication Flow

1. ESP32 publishes sensor data to micro-ROS agent (UDP port 9999)

2. micro-ROS agent translates to ROS 2 topics
3. ROS 2 subscribers receive sensor messages
4. (Optionally) ROS 2 commands published back to ESP32

3.4 Data Types

- `sensor_msgs/msg/Imu`: 6-axis IMU data
- `sensor_msgs/msg/Range`: Ultrasonic distance measurements

Chapter 4

Troubleshooting

4.1 Common Issues and Solutions

4.1.1 Docker Compose Issues

Problem: Port 9999 already in use

```
# Kill any existing micro-ROS processes
killall -9 micro_ros_agent 2>/dev/null

# Or change port in docker-compose.yaml
nano docker-compose.yaml
```

Problem: Docker image build fails

```
# Clean and rebuild
docker compose down
docker system prune -f
docker compose build --no-cache
docker compose up
```

4.1.2 ESP32 Firmware Issues

Problem: I2C communication fails with MPU6050

- Verify pull-up resistors (4.7kΩ recommended) on SCL/SDA lines
- Check connections: GPIO 22 (SCL), GPIO 21 (SDA)
- Confirm I2C address: 0x68 (check with `i2cdetect`)
- Reduce I2C frequency if needed

Problem: Ultrasonic sensor readings are incorrect

- Verify GPIO 9 (Trigger) and GPIO 10 (Echo) are connected
- Check power supply voltage (5V needed for HC-SR04)
- Ensure no interference from other GPIO operations
- Test with a simple GPIO toggle program first

Problem: ESP32 won't flash

```
# Check USB connection
ls -l /dev/ttyUSB*
```

```
# Erase flash completely
idf.py erase-flash

# Try again
idf.py flash
```

4.1.3 micro-ROS Connection Issues

Problem: ESP32 can't connect to WiFi

- Verify SSID and password in `app.c`
- Check WiFi network is accessible from ESP32 location
- Enable serial monitor to see debug output:

```
idf.py monitor
```

Problem: micro-ROS topics not appearing on host

- Verify micro-ROS agent is running:

```
docker compose ps
```

- Check port 9999 is open:

```
netstat -tuln | grep 9999
```

- Monitor agent logs:

```
docker compose logs micro_ros_agent
```

- Verify ESP32 can reach host (test ping from ESP32)

4.1.4 ROS 2 Build Issues

Problem: Package dependencies not found

```
# Inside workspace container
rosdep update
rosdep install --from-paths src --ignore-src -y
```

Problem: colcon build fails

```
# Clean and rebuild
rm -rf build install log
colcon build
```

4.1.5 Documentation Build Issues

Problem: Sphinx documentation won't build

```
# Inside workspace container
cd docs
sphinx-build -W -b html source _build/html
```

4.2 Using Debug Tools

4.2.1 Serial Monitor (ESP32)

```
idf.py monitor -p /dev/ttyUSB0 -b 115200
```

4.2.2 ROS 2 Message Inspection

```
# In host ROS 2 environment
ros2 topic list
ros2 topic echo /imu/data
ros2 topic echo /range/data
```

4.3 Getting Help

1. Check serial output from ESP32 for error messages
2. Review micro-ROS agent logs
3. Verify hardware connections with multimeter
4. Check ROS 2 node graph: `rqt_graph`